**CLOUD SECURITY SPECIALIST**
**NAME:KEMUNTO FAITH OGWOKA**
**CS NO:ADC-CSS02-25071**
**GMAIL:faithogwoka@gmail.com**

# LAB:KEY VAULT (IMPLEMENTING SECURE DATA BY SETTING UP ALWAYS ENCRYPTED)

## 1.1 INTRODUCTION

I started the lab with a required lab set up.I clicked on ctrl alt and delete to bring up the login page,signed in as admin and then downloaded the lab files.

## 1.2 PREPARE CLOUD SHELL FOR LATER USE

**Step 1**

I logged in to the Azure portal using the credentials provided

**Step 2**

In the toolbar at the top of the Azure portal, I selected the Cloud Shell icon.

**Step 3**

In the Welcome to Azure Cloud Shell dialog, I selected PowerShell.

**Step 4**

On the Getting Started screen I selected Mount Storage Account.

**Step 5**

In the Storage account subscription drop down, I selected my subscription then the apply button.

**Step 6**

In the Mount Storage Account screen,I selected I want to create a storage account.

**Step 7**

On the Create Storage Account screen, I filled in the fields for;resource group,storage account,file share,region, then clicked on Create button.

**Step 8**

After the cloud shell initialized and put me at a text prompt, I exited the shell.

## 1.3 LAB 07: KEY VAULT (IMPLEMENTING SECURE DATA BY SETTING UP ALWAYS ENCRYPTED)

**LAB SCENARIO**

I was asked to create a proof of concept application that makes use of the Azure SQL Database support for Always Encrypted functionality. All of the secrets and keys used in this scenario should be stored in Key Vault. The application should be registered in Microsoft Entra ID in order to enhance its security posture. To accomplish these objectives, the proof of concept should include:Creating an Azure Key Vault and storing keys and secrets in the vault and Creating a SQL Database and encrypting content of columns in database tables by using Always Encrypted.

## EXERCISE 1:DEPLOY THE BASE INFRASTRUCTURE FROM AN ARM TEMPLATE

In this exercise ,I completed the task;Task 1: Deploy an Azure VM and an Azure SQL database

## TASK 1:DEPLOY AN AZURE VM AND AN AZURE SQL DATABASE

In this task, l deployed  an Azure VM, which will automatically install Visual Studio 2019 and SQL Server Management Studio 19 as part of the deployment.

**Step 1**

I signed in to the Azure portal

**Step 2**

In the Azure portal, in the Search resources, services, and docs text box at the top of the Azure portal page, I typed Deploy a custom template and pressed the Enter key.

**Step 3**

On the Custom deployment blade, I clicked the Build your own template in the editor option.

**Step 4**

On the Edit template blade, I clicked on load file, and located the \Allfiles\Labs\10\az-500-10_azuredeploy.json file and clicked Open.

**Step 5**

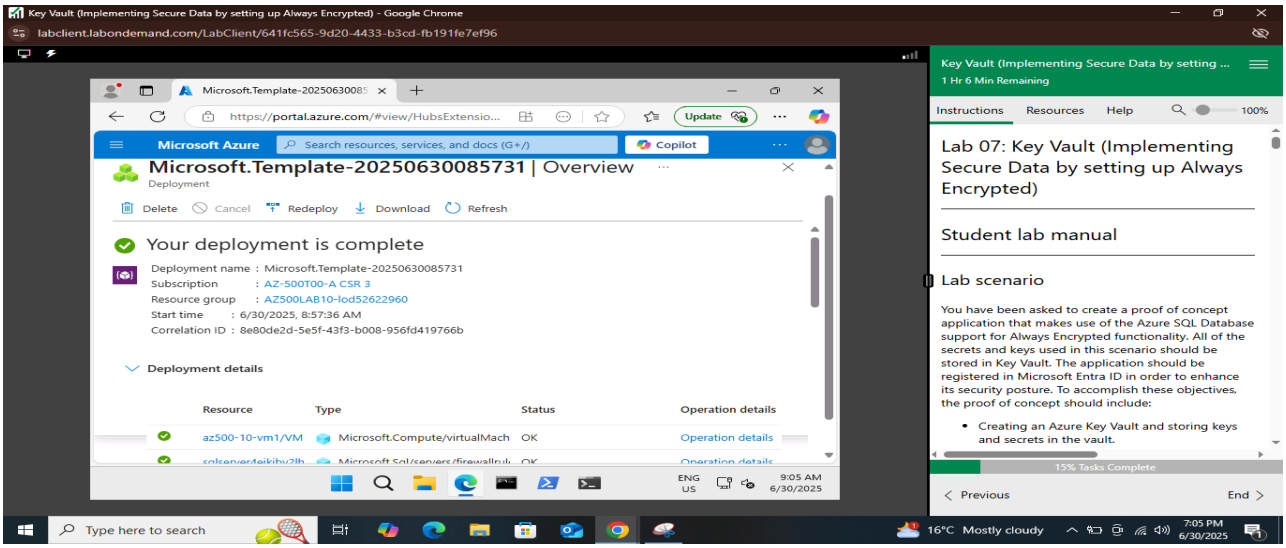
On the Edit template blade, I clicked on  Save.

**Step 6**

On the Custom deployment blade, under Deployment Scope I ensured that the settings provided for subscription,resource group,location,username and password were configured and left any others with their default values.

**Step 7**

I Clicked the Review and Create button, and confirmed the deployment by clicking the Create button.Here is my screenshot



## EXERCISE 2:CONFIGURE THE KEY VAULT RESOURCE WITH A KEY AND A SECRET

In this exercise I completed the tasks:Task 1: Create and configure a Key Vault,Task 2: Add a key to the Key Vault and Task 3: Add a secret to the Key Vault.

### TASK 1:CREATE AND CONFIGURE A KEY VAULT

In this task,I created an Azure Key Vault resource and configured the Azure Key Vault permissions.

**Step 1**

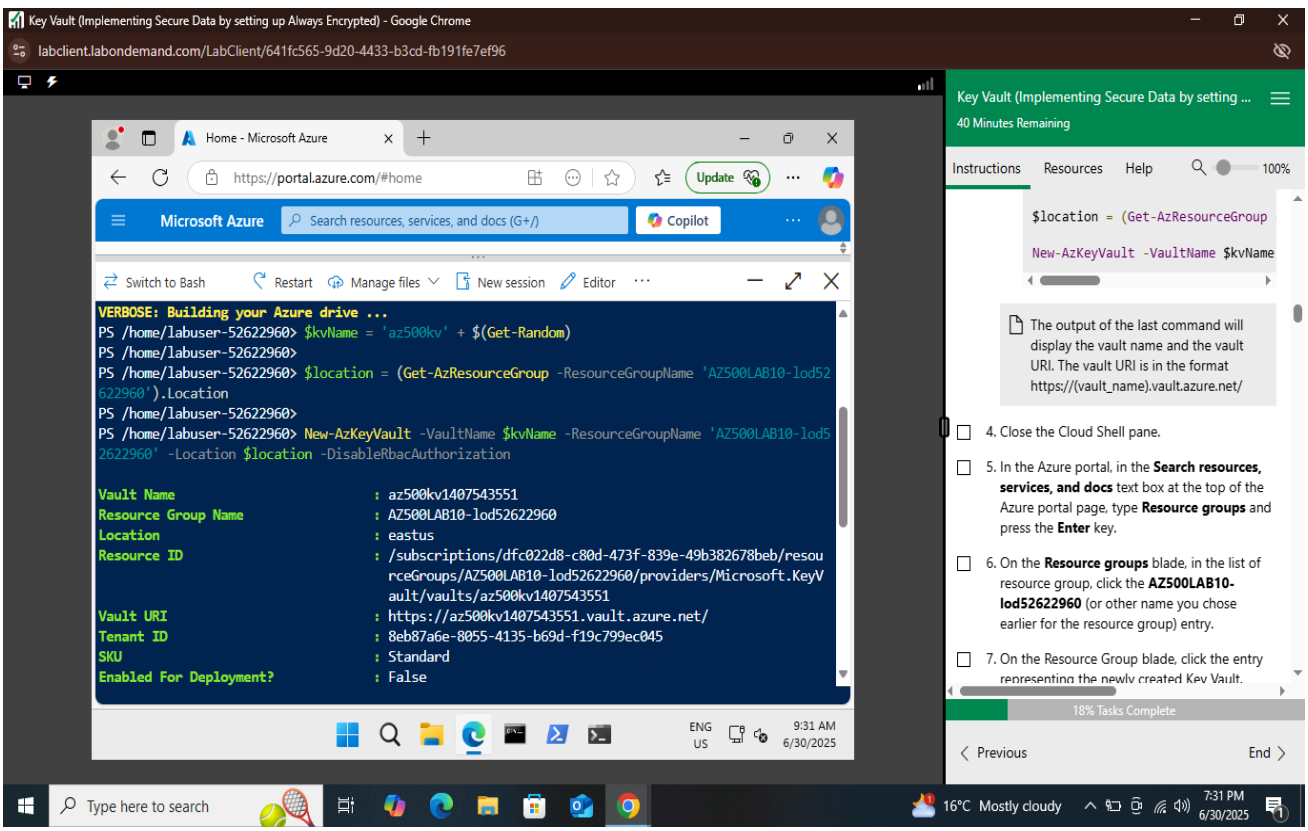
I opened the Cloud Shell by clicking the first icon next to the search bar at the top right of the Azure portal. When prompted, I selected PowerShell and Created storage.

**Step 2**

I ensured that PowerShell was selected in the drop-down menu in the upper-left corner of the Cloud Shell pane.

**Step 3**

In the PowerShell session within the Cloud Shell pane,I ran the command provided to create an Azure Key Vault in the resource group AZ500LAB10-lod52622960.Here is my screenshot




**Step 4**

I closed the cloud shell pane

**Step 5**

In the Azure portal, in the Search resources, services, and docs text box at the top of the Azure portal page, I typed Resource groups and pressed the Enter key.

**Step 6**

On the Resource groups blade, in the list of resource groups, I clicked the AZ500LAB10-lod52622960 entry.

**Step 7**

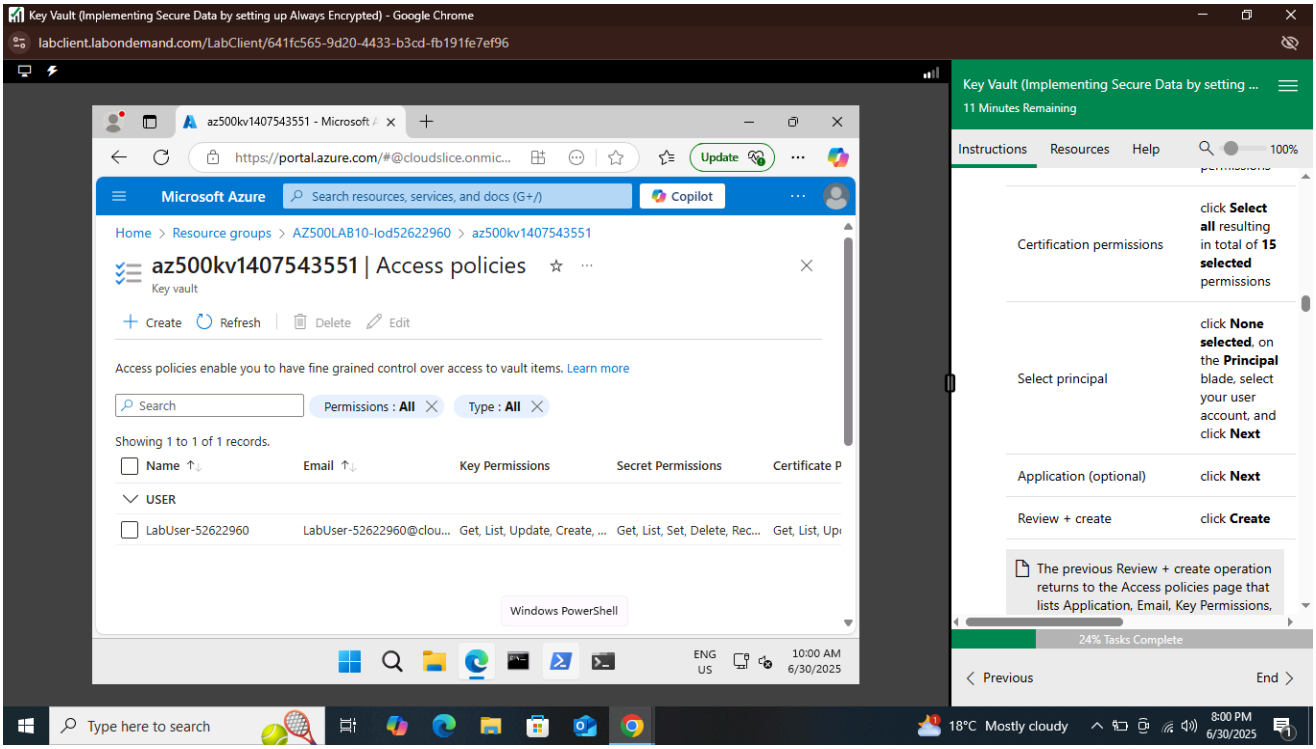
I clicked the entry representing the newly created Key Vault.

**Step 8**

On the Key Vault blade, in the Overview section, I clicked on Access Policies and then clicked + Create.

**Step 9**

On the Create an access policy blade, I specified the settings provided and left others as default.Here is my screenshot




## TASK 2:ADD A KEY TO KEY VAULT

In this task,I added a key to the Key Vault and viewed information about the key.

**Step 1**

In the Azure portal, I opened a PowerShell session in the Cloud Shell pane.

**Step 2**

I ensured PowerShell was selected in the upper-left drop-down menu of the Cloud Shell pane.

**Step 3**

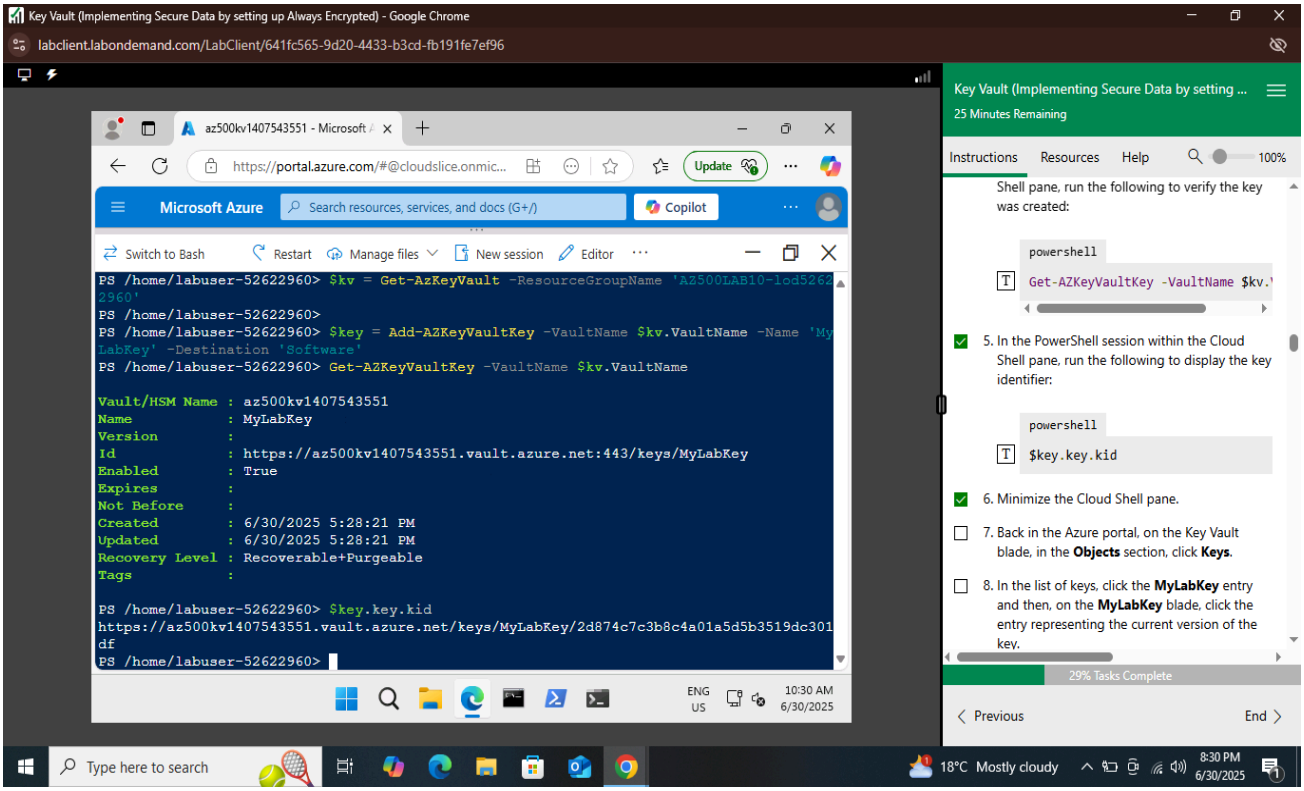
In the PowerShell session within the Cloud Shell pane, I ran the command provided to add a software-protected key to the Key Vault.

**Step 4**

In the PowerShell session within the Cloud Shell pane,I ran the command;Get-AZKeyVaultKey -VaultName $kv.VaultName to verify the key was created

**Step 5**

In the PowerShell session within the Cloud Shell pane, I ran; $key.key.kid to display the key identifier.Here is my screenshot for steps 3,4 and 5




**Step 6**

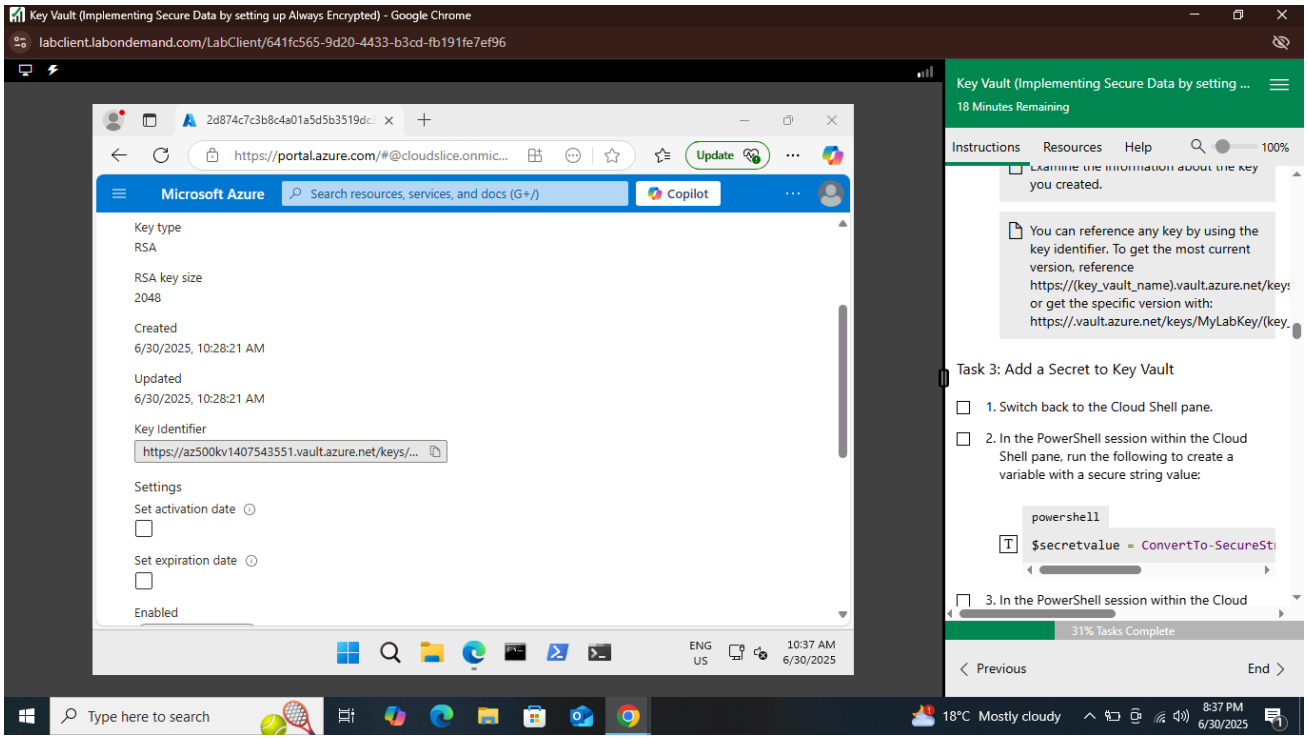
I minimized the Cloud Shell pane.

**Step 7**

Back in the Azure portal, on the Key Vault blade, in the Objects section, I clicked on Keys.

**Step 8**

In the list of keys, I clicked the MyLabKey entry and then, on the MyLabKey blade, clicked the entry representing the current version of the key.I examined the information about the key I created.Below is my screenshot




## TASK 3:ADD A SECRET TO KEY VAULT

**Step 1**

I switched back to the Cloud Shell pane.

**Step 2**

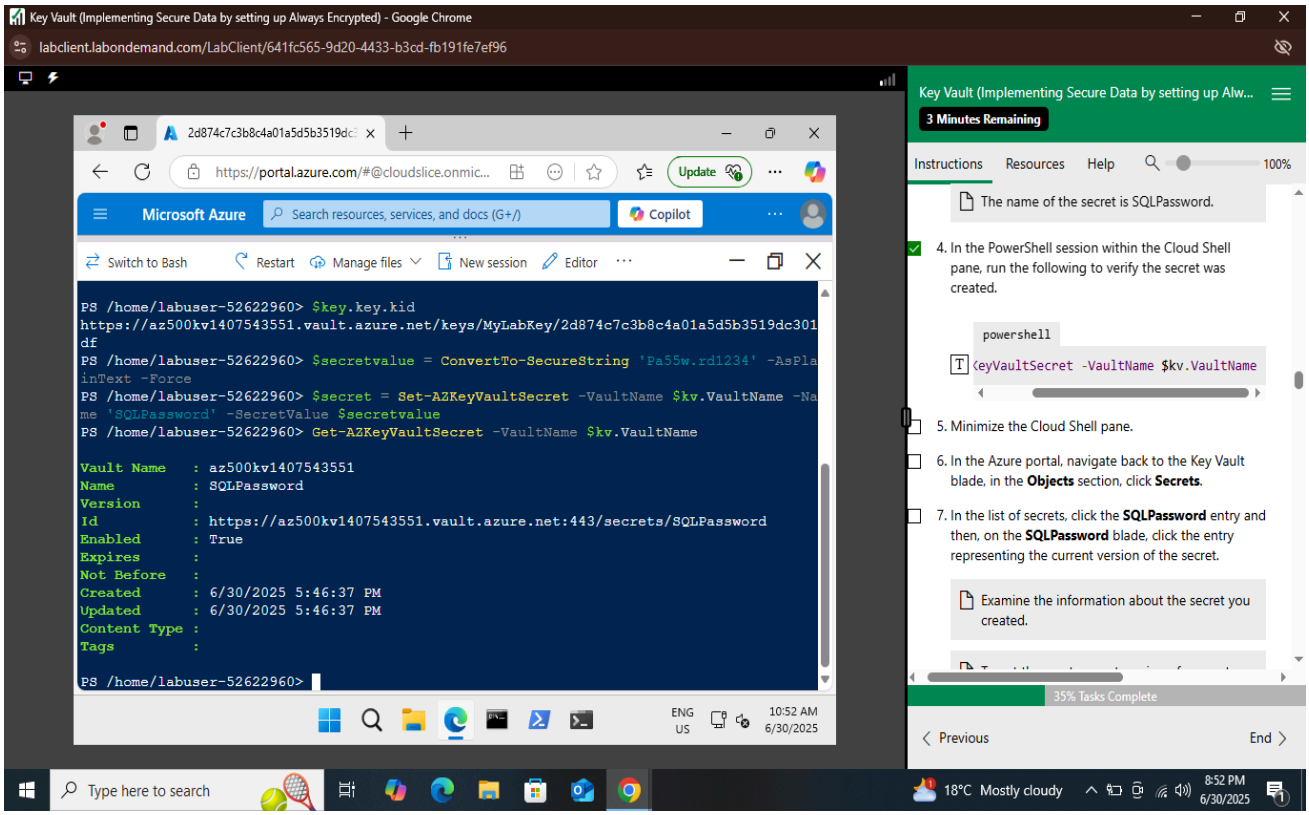
In the PowerShell session within the Cloud Shell pane, I ran the following; '$secretvalue = ConvertTo-SecureString 'Pa55w.rd1234' -AsPlainText -Force' to create a variable with a secure string value.

**Step 3**

In the PowerShell session within the Cloud Shell pane, I ran the following;'$secret = Set-AZKeyVaultSecret -VaultName $kv.VaultName -Name 'SQLPassword' -SecretValue $secretvalue' to add the secret to the vault.

**Step 4**

In the PowerShell session within the Cloud Shell pane, I ran the following; 'Get-AZKeyVaultSecret -VaultName $kv.VaultName' to verify the secret was created.Below is my screenshot for steps 2,3 and 4




**Step 5**

I minimized the cloud shell pane

**Step 6**

In the Azure portal, I navigate back to the Key Vault blade, in the Objects section, clicked Secrets.

**Step 7**

In the list of secrets, I clicked the SQLPassword entry and then, on the SQLPassword blade, clicked the entry representing the current version of the secret and examined the information about the secret I created.

## EXERCISE 3:CONFIGURE AN AZURE SQL DATABASE AND A DATA- DRIVEN APPLICATION

In this exercise,I completed the following tasks;Task 1: Enable a client application to access the Azure SQL Database service,Task 2: Create a policy allowing the application access to the Key Vault,Task 3: Retrieve SQL Azure database ADO.NET Connection String,Task 4: Log on to the Azure VM running Visual Studio 2019 and SQL Management Studio 19 and Task 5: Create a table in the SQL Database and select data columns for encryption.

# TASK 1:ENABLE A CLIENT APPLICATION TO ACCESS THE AZURE SQL DATABASE SERVICE

In this task, I enabled a client application to access the Azure SQL Database service. This was done by setting up the required authentication and acquiring the Application ID and Secret that I needed to authenticate my application.

**Step 1**

In the Azure portal, in the Search resources, services, and docs text box at the top of the Azure portal page,I typed App Registrations and pressed the Enter key.

**Step 2**

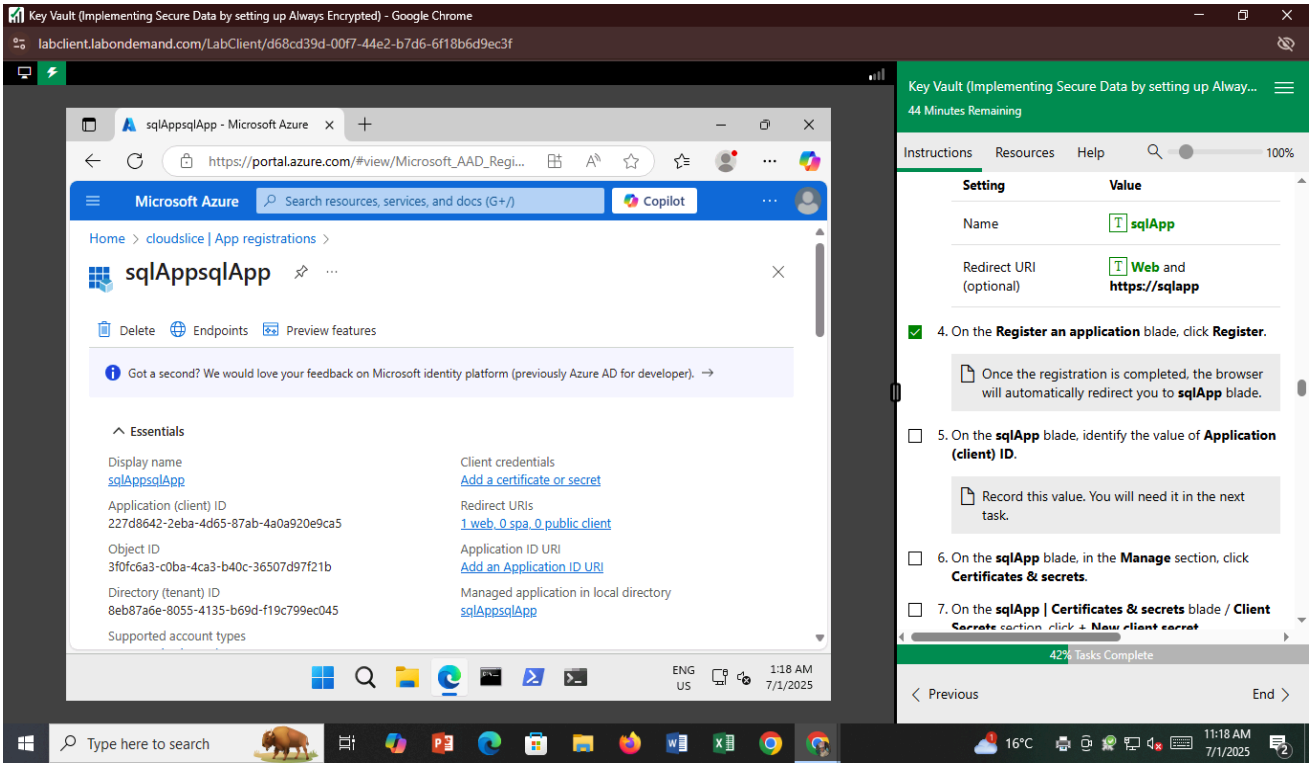
On the App Registrations blade, I clicked + New registration.

**Step 3**

On the Register an application blade,I specified the settings for name and redirect url.

**Step 4**

On the Register an application blade, I clicked on Register.Once the registration was completed, the browser automatically redirected me to sqlApp blade.Here is my screenshot




**Step 5**

On the sqlApp blade, I identified the value of Application (client) ID and recorded it for later use.

**Step 6**

On the sqlApp blade, in the Manage section, I clicked Certificates & secrets.

**Step 7**

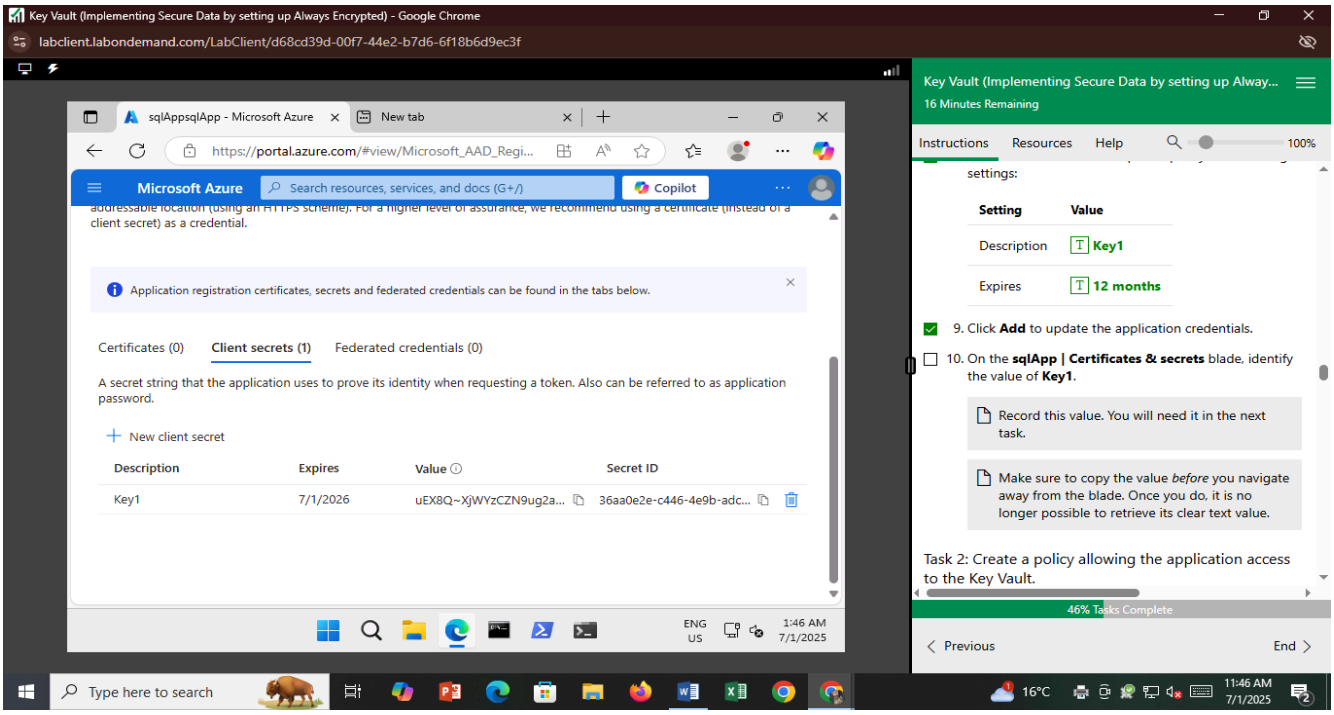
On the sqlApp | Certificates & secrets blade / Client Secrets section, I clicked + New client secret.

**Step 8**

In the Add a client secret pane,I  specified the settings provided for description and expiry.

**Step 9**

I clicked on  Add to update the application credentials.Here is my screenshot




**Step 10**

On the sqlApp | Certificates & secrets blade,I identified the value of Key1.

## TASK 2:CREATE A POLICY ALLOWING THE APPLICATION ACCESS TO THE KEY VAULT

In this task, I granted the newly registered app permissions to access secrets stored in the Key Vault.

**Step 1**

In the Azure portal, I opened a PowerShell session in the Cloud Shell pane.

**Step 2**

I ensured PowerShell was selected in the upper-left drop-down menu of the Cloud Shell pane.

**Step 3**

In the PowerShell session within the Cloud Shell pane, I ran the command; $applicationId = '<Azure_AD_Application_ID>' to create a variable storing the Application (client) ID.I replaced the (Azure_AD_Application_ID) placeholder with the value of the Application (client) ID) I recorded earlier.

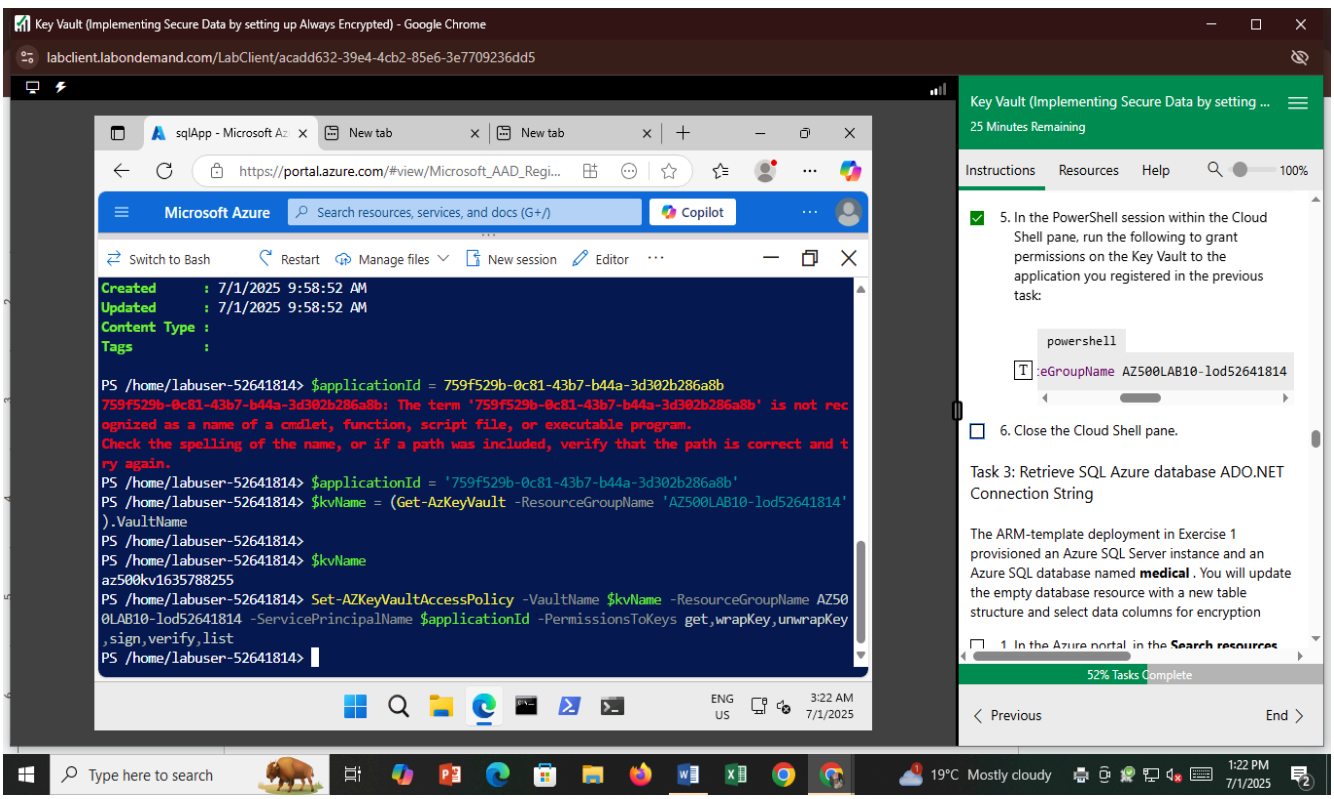
**Step 4**

In the PowerShell session within the Cloud Shell pane, I ran the command; $kvName = (Get-AzKeyVault -ResourceGroupName 'AZ500LAB10-lod52641814').VaultName
$kvName to create a variable storing the Key Vault name.

**Step 5**

In the PowerShell session within the Cloud Shell pane,I ran the command; 'Set-AZKeyVaultAccessPolicy -VaultName $kvName -ResourceGroupName AZ500LAB10-lod52641814 -ServicePrincipalName $applicationId -PermissionsToKeys get,wrapKey,unwrapKey,sign,verify,list' to grant permissions on the Key Vault to the application I registered in the previous task.

Here is my screenshot for steps 3,4 and 5




**Step 6**

I closed the cloud pane

## TASK 3:RETRIEVE SQL AZURE DATABASE ADO.NET CONNECTION STRING

The ARM-template deployment in Exercise 1 provisioned an Azure SQL Server instance and an Azure SQL database named medical . I updated the empty database resource with a new table structure and selected data columns for encryption.

**Step 1**

In the Azure portal, in the Search resources, services, and docs text box at the top of the Azure portal page,I typed SQL databases and pressed the Enter key.

**Step 2**

In the list of SQL databases, I clicked the medical() entry.

**Step 3**

On the SQL database blade, in the Settings section, I clicked on Connection strings.

**Step 4**

I recorded the ADO.NET (SQL authentication) connection string for later.

## TASK 4:LOG ON TO THE AZURE VM RUNNING VISUAL STUDIO 2019 AND SQL MANAGEMENT STUDIO 19

In this task, I logged on to the Azure VM, which deployment initiated in Exercise 1. This Azure VM hosts Visual Studio 2019 and SQL Server Management Studio 19.

**Step 1**

In the Azure portal, in the Search resources, services, and docs text box at the top of the Azure portal page, I typed virtual machines and pressed the Enter key.

**Step 2**

In the list of Virtual Machines shown,I selected the az500-10-vm1 entry. On the az500-10-vm1 blade, on the Essentials pane, took note of the Public IP address to use later.

## TASK 5:CREATE A TABLE IN THE SQL DATABASE AND SELECT DATA COLUMNS FOR ENCRYPTION

In this task,I connected to the SQL Database with SQL Server Management Studio and created a table. I then encrypted two data columns using an auto generated key from the Azure Key Vault.

**Step 1**

In the Azure portal, I navigated to the blade of the medical SQL database, in the Essentials section, identified the Server name copied to clipboard, and then, in the toolbar, clicked on Set server firewall.

**Step 2**

On the Firewall settings blade, I scrolled down to Rule Name, clicked on + Add a firewall rule, and specified the settings provided.

**Step 3**

I clicked the Save button to save the change and closed the confirmation pane.

**Step 4**

I navigated back to the az500-10-vm1 blade, clicked on Overview, Connect and, in the drop down menu, clicked Connect.

**Step 5**

I downloaded the RDP file and used it to connect to the az500-10-vm1 Azure VM via Remote Desktop. When prompted to authenticate, provided credentials for username and password.

**Step 6**

I installed SQL Server Management Studio on az500-10-vm1. Azure VM.

**Step 7**

I opened SQL Server Management Studio.

**Step 8**

In the Connect to Server dialog box, I specified the settings provided.

**Step 9**

In the Connect to Server dialog box, I clicked on Connect.

**Step 10**

Within the SQL Server Management Studio console, in the Object Explorer pane, I expanded the Databases folder.

**Step 11**

In the Object Explorer pane, I right-clicked the medical database and clicked New Query.

**Step 12**

I pasted the code provided into the query window and clicked Execute. This creates a Patients table.

**Step 13**

After the table is created successfully, in the Object Explorer pane, I expanded the medical database node, the tables node, right-clicked the dbo.Patients node, and clicked Encrypt Columns.

**Step 14**

On the Introduction page,I clicked Next.

**Step 15**

On the Column Selection page, I selected the SSN and Birthdate columns, set the Encryption Type of the SSN column to Deterministic and of the Birthdate column to Randomized, and clicked Next.

**Step 16**

On the Master Key Configuration page, I selected Azure Key Vault, clicked Sign in, when prompted, authenticated by using the same user account I used to provision the Azure Key Vault instance earlier in the lab, ensured that Key Vault appears in the Select an Azure Key Vault drop down list, and clicked Next.

**Step 17**

On the Run Settings page,I clicked Next.

**Step 18**

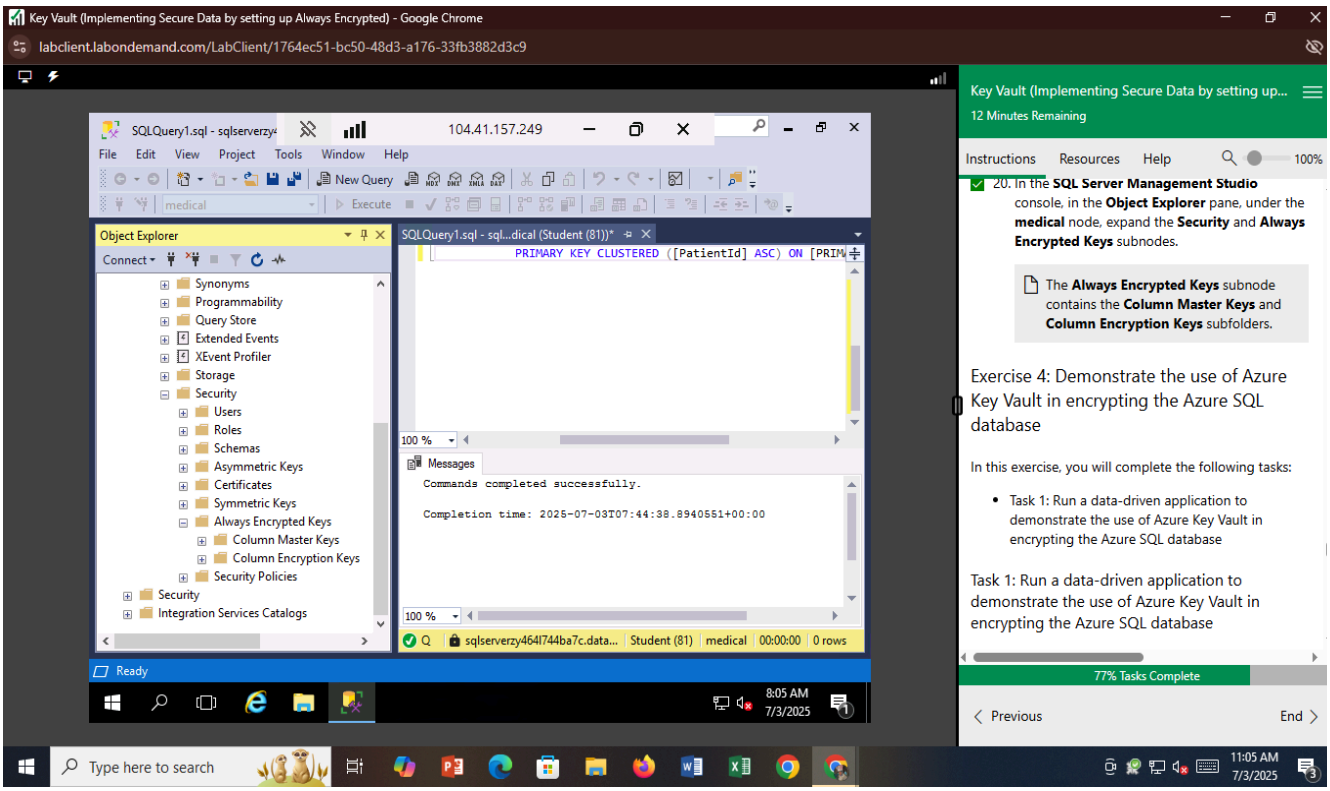
On the Summary page, I clicked Finish to proceed with the encryption.I was prompted to sign in again.

**Step 19**

Once the encryption process was complete, on the Results page,I  clicked Close.

**Step 20**

In the SQL Server Management Studio console, in the Object Explorer pane, under the medical node, I expanded the Security and Always Encrypted Keys subnodes.The Always Encrypted Keys subnode contains the Column Master Keys and Column Encryption Keys subfolders.Here is my screenshot.




## EXERCISE 4:DEMONSTRATE THE USE OF AZURE KEY VAULT IN ENCRYPTING THE AZURE SQL DATABASE

In this exercise, I completed;Task 1: Run a data-driven application to demonstrate the use of Azure Key Vault in encrypting the Azure SQL database.

### TASK 1:RUN A DATA-DRIVEN APPLICATION TO DEMONSTRATE THE USE OF AZURE KEY VAULT IN ENCRYPTING THE AZURE SQL DATABASE

In this task;I created a Console application using Visual Studio to load data into the encrypted columns and then access that data securely using a connection string that accesses the key in the Key Vault.

**Step 1**

From the RDP session to the az500-10-vm1,I launched Visual Studio 2019 from the Start menu.

**Step 2**

I switched to the window displaying Visual Studio 2019 welcome message, clicked the Sign in button and, when prompted, provided the credentials I used to authenticate to the Azure subscription in the lab.

**Step 3**

On the Get started page, I clicked on Create a new project.

**Step 4**

In the list of project templates, I searched for Console App (.NET Framework), in the list of results, clicked Console App (.NET Framework) for C#, and clicked Next.

**Step 5**

On the Configure your new project page, I specified the settings for project and solution name and left others with their default values, then clicked OK.

**Step 6**

In the Visual Studio console, I clicked the Tools menu, in the drop down menu, clicked NuGet Package Manager, and, in the cascading menu, clicked Package Manager Console.

**Step 7**

In the Package Manager Console pane, I ran the command 'Install-Package Microsoft.SqlServer.Management.AlwaysEncrypted.AzureKeyVaultProvider' to install the first required NuGet package.

**Step 8**

In the Package Manager Console pane, I ran the command 'Install-Package Microsoft.IdentityModel.Clients.ActiveDirectory' to install the second required NuGet package.

**Step 9**

I minimized the RDP session to my Azure virtual machine, then navigated to \Allfiles\Labs\10\program.cs, opened it in Notepad,and copied its content into Clipboard.

**Step 10**

I returned to the RDP session, and in the Visual Studio console, in the Solution Explorer window, clicked Program.cs and replaced its content with the code I copied into Clipboard.

**Step 11**

In the Visual Studio window, in the Program.cs pane, in line 15,I replaced the connection string placeholder with the Azure SQL database ADO.NET connection string I recorded earlier in the lab.

**Step 12**

In the Visual Studio window, in the Program.cs pane, in line 16, I replaced the client id n placeholder with the value of Application (client) ID of the registered app I recorded earlier in the lab.

**Step 13**

In the Visual Studio window, in the Program.cs pane, in line 17, I replaced the key value placeholder with the value of Key1 of the registered app I recorded earlier in the lab.

**Step 14**

In the Visual Studio console, I clicked the Start button to initiate the build of the console application and started it.

**Step 15**

The application starts a Command Prompt window. When prompted for password, I typed the password I specified in the deployment in Exercise 1 to connect to Azure SQL Database.

**Step 16**

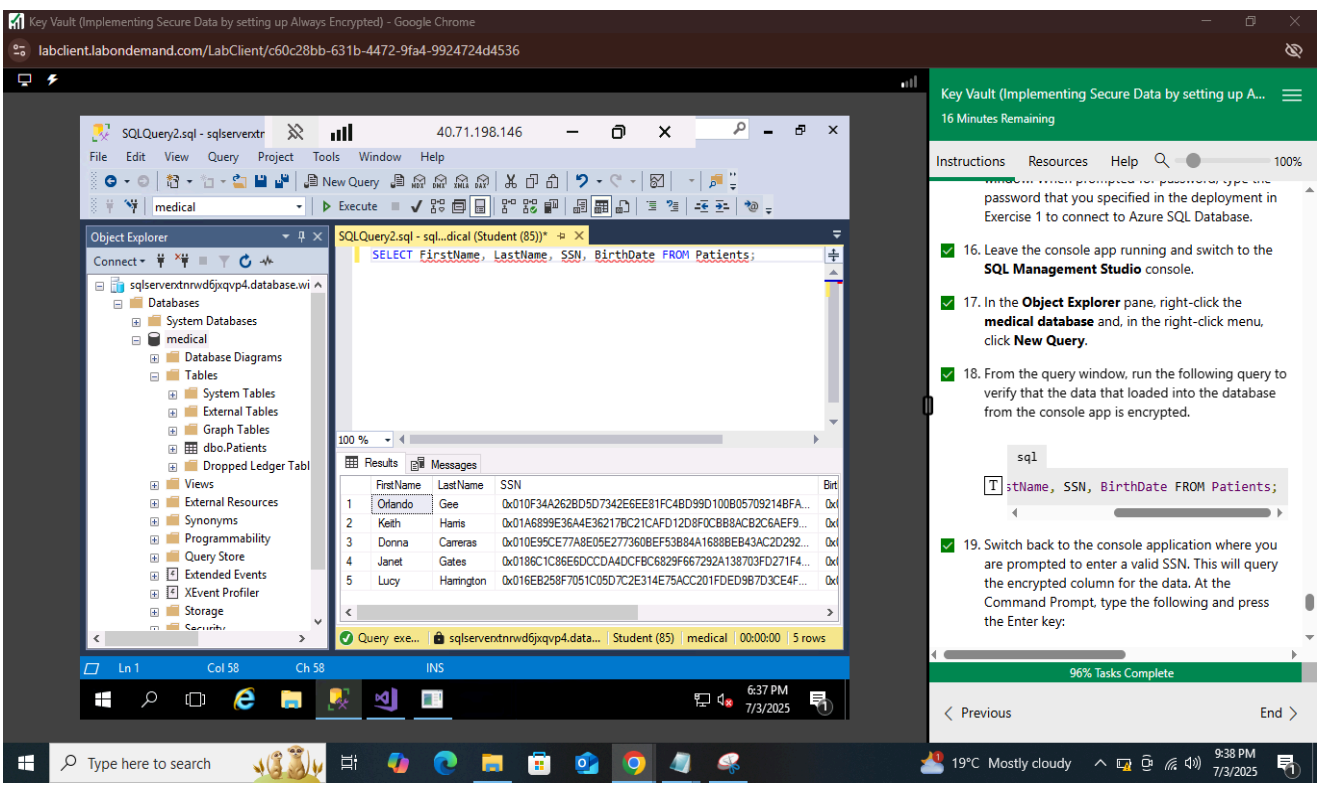
I left the console app running and switched to the SQL Management Studio console.

**Step 17**

In the Object Explorer pane, I right-clicked the medical database and, in the right-click menu, clicked New Query.

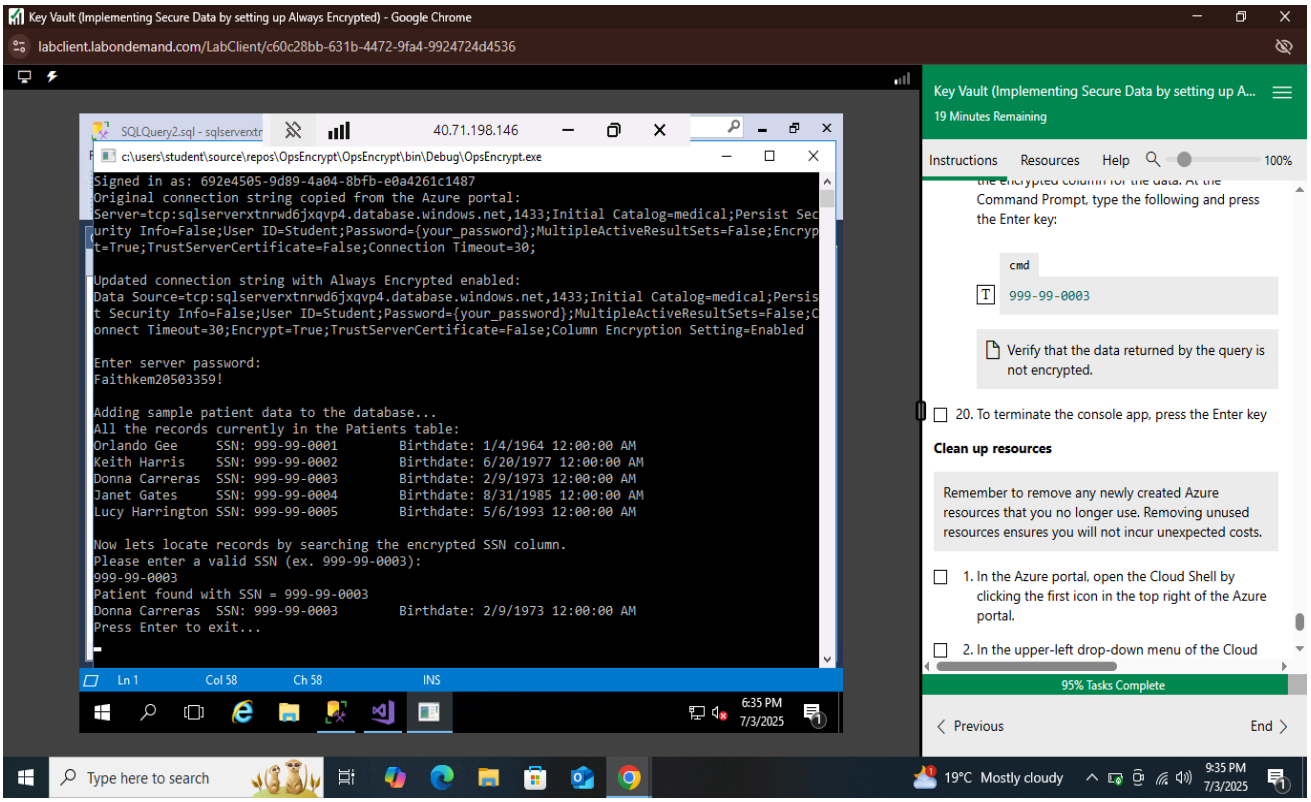
**Step 18**

From the query window, I ran the following query 'SELECT FirstName, LastName, SSN, BirthDate FROM Patients;' to verify that the data that loaded into the database from the console app is encrypted.Here is my screenshot



## Step 19

I Switched back to the console application where I was prompted to enter a valid SSN. This will query the encrypted column for the data. At the Command Prompt,I typed 999-99-0003 and pressed the Enter key.I verified that the data returned by the query is not encrypted. Here is my screenshot




## Step 20

I pressed the Enter key to terminate the console app.I then cleaned up resources and completed the lab.